CineControl

RELATÓRIO FINAL DE TESTES

Versão 2.0 — Revisada

Trabalho Final — Teste de Software

UCB – Universidade Católica de Brasília

Tema: Cinema

Data: Outubro/2026

Integrantes:

Victor Emanoel · André Luiz · Paulo dos Santos · Iasmin Linhares · Igor Ferreira

1. Introdução

Este documento é a entrega revisada (v2.0) do Trabalho Final da disciplina de Teste de Software, referente ao sistema CineControl. Ele incorpora as correções indicadas pelo feedback docente e reúne o Plano de Testes, os Casos de Teste, a Implementação dos Testes, as Evidências de Execução (incluindo falhas propositais que evidenciam defeitos conhecidos) e o Relatório Final exigido pelo roteiro da disciplina.

O Documento de Visão e as Histórias de Usuário do CineControl (com critérios de aceitação e cenários BDD) foram entregues em arquivos próprios e são referenciados ao longo deste relatório quando necessário.

1.1 Sobre o Sistema

O CineControl é um sistema acadêmico desenvolvido em Python para gerenciamento das atividades básicas de um cinema: cadastro e consulta de filmes, salas e sessões, e reserva de ingressos por clientes. O sistema é composto por:

- Interface de linha de comando (CLI) — frontend de interação com o usuário, implementado em cli.py.
- Backend com API REST (Flask) — camada de comunicação entre a CLI e as regras de negócio, implementada em api.py.
- Módulo de regras de negócio — lógica central do sistema, implementada em core.py.
- Armazenamento em memória — o protótipo não utiliza banco de dados real; os dados são mantidos em estruturas Python em tempo de execução, sendo reiniciados a cada execução do sistema.

Esta arquitetura — CLI + API REST + dados em memória — é a realidade do protótipo entregue e é o objeto de todos os testes descritos neste documento. Referências anteriores a "sistema web" ou "banco de dados" foram revisadas nesta versão para alinhar o texto à implementação real.

1.2 Objetivo deste Documento

Apresentar de forma organizada tudo o que é exigido pelo roteiro da disciplina nas etapas 4.2 a 4.5 (Planejamento de Testes, Casos de Teste, Implementação dos Testes, Execução e Evidências, e Relatório Final), com base no sistema CineControl e em sua documentação de requisitos já entregue.

2. Plano de Testes

2.1 Objetivo dos Testes

Validar, por meio de testes unitários, testes de API e testes E2E, se as funcionalidades do CineControl atendem aos requisitos funcionais (RF01–RF24) e às regras de negócio (RN01–RN12) definidos no Documento de Visão, com foco nas Histórias de Usuário HU01 (Login), HU02 (Gerenciar Filmes), HU03 (Gerenciar Salas), HU04 (Gerenciar Sessões), HU06 (Realizar Reserva) e HU08 (Cancelar Reserva).

2.2 Escopo do Teste

O que será testado:

- Autenticação de usuários (login com credenciais válidas e inválidas) — HU01.
- Cadastro, consulta e exclusão de filmes, incluindo a regra de bloqueio de exclusão com sessões vinculadas — HU02.
- Cadastro e exclusão de salas, incluindo validação de capacidade — HU03.
- Cadastro de sessões, incluindo o cálculo automático de ingressos disponíveis e a validação de conflito de horário — HU04.
- Consulta de programação (filmes e sessões disponíveis) — HU05.
- Realização de reservas, incluindo validação de quantidade e de disponibilidade — HU06.
- Consulta das reservas do cliente autenticado — HU07.
- Cancelamento de reservas e devolução de ingressos à disponibilidade da sessão — HU08.

O que não será testado:

- Persistência em banco de dados real (o protótipo usa armazenamento em memória, reiniciado a cada execução).
- Pagamento online, integração com cartão/Pix e emissão de nota fiscal (fora do escopo, conforme Documento de Visão, seção 5.2).
- Testes de carga, performance e segurança (não exigidos pelo roteiro da disciplina).
- Interface gráfica — o frontend é uma CLI; a interação é validada via API.

2.3 Ferramentas Utilizadas

Tipo de Teste	Ferramenta / Framework	Justificativa
Testes unitários	unittest (Python stdlib)	Dispensa dependências externas; equivalente ao PyTest para o escopo do projeto.
Testes de API	unittest + Flask test client	Testa endpoints REST sem subir um servidor HTTP real; equivalente ao Postman/Supertest.
Testes E2E	unittest + Flask test client	Simula a jornada completa do usuário via API, cobrindo fluxos de administrador, cliente e cenários de erro.
Controle de versão	Git / GitHub	Versionamento do código-fonte, testes e documentação.

2.4 Estratégia de Testes

A estratégia adotada segue a pirâmide de testes: uma base ampla de testes unitários cobrindo as regras de negócio isoladamente (módulo core.py), testes de API cobrindo os endpoints REST expostos pelo backend (módulo api.py), e três testes E2E cobrindo os fluxos de ponta a ponta mais importantes do sistema: o fluxo do administrador (cadastro de filmes, salas e sessões), o fluxo completo do cliente (reserva e cancelamento) e um fluxo de cenário de erro (tentativa de ações inválidas).

Critério de sucesso: para cada Caso de Teste, o critério de sucesso está definido explicitamente na coluna "Critério de Sucesso" da seção 3. Um teste é considerado "Passou" quando o resultado obtido corresponde ao critério definido, e "Falhou" quando diverge — inclusive quando a divergência evidencia um defeito conhecido (ver seção 5).

2.5 Papéis e Responsabilidades

Integrante	Responsabilidade
Victor Emanoel	Documentação de requisitos (Documento de Visão), revisão geral do Plano de Testes e controle de versionamento no GitHub.
André Luiz	Implementação do código-fonte (core.py, api.py, cli.py) e dos testes unitários (CT-U01 a CT-U15).
Paulo dos Santos	Implementação dos testes de API (CT-A01 a CT-A09) e do primeiro teste E2E (CT-E01); execução e coleta de evidências.
Iasmin Linhares	Histórias de Usuário, elaboração dos Casos de Teste e consolidação do Relatório Final.
Igor Ferreira	Implementação dos testes E2E adicionais (CT-E02 e CT-E03), revisão dos casos de teste e contribuição para o repositório GitHub.

3. Casos de Teste

Cada caso de teste segue o formato exigido pelo roteiro: Identificador, Descrição, Passos numerados para execução, Dados de entrada/saída esperada, Critério de Sucesso explícito e Resultado da última execução. As evidências completas estão na seção 5.

3.1 Casos de Teste Unitários

Framework: unittest. Arquivo: tests/test_unit.py.

ID	HU / Regra	Descrição e Passos	Entrada / Saída Esperada	Critério de Sucesso	Resultado
CT-U01	HU01/RF01	1. Instanciar CineControl. 2. Chamar autenticar(email, senha) com credenciais do admin. 3. Verificar o perfil retornado.	Entrada: admin@cinecontrol.com / admin123 Saída esperada: objeto usuário com perfil="admin"	Retorno com perfil="admin" sem exceção.	**Passou**
CT-U02	HU01/RN01	1. Instanciar CineControl. 2. Chamar autenticar() com senha incorreta. 3. Verificar se ErroValidacao é lançado.	Entrada: admin@cinecontrol.com / senha_errada Saída esperada: exceção ErroValidacao	ErroValidacao levantado com mensagem de credenciais inválidas.	**Passou**
CT-U03	HU01/RN01	1. Instanciar CineControl. 2. Chamar autenticar() com senha em caixa diferente (ADMIN123). 3. Verificar se ErroValidacao é lançado.	Entrada: admin@cinecontrol.com / ADMIN123 Saída esperada: ErroValidacao (senha é case-sensitive)	ErroValidacao levantado — a senha com caixa diferente deve ser rejeitada.	**Falhou (Bug 1)**
CT-U04	HU02/RF03	1. Instanciar CineControl. 2. Chamar cadastrar_filme(titulo="Duna", duracao=155). 3. Listar filmes e verificar presença do novo filme.	Entrada: título="Duna", duração=155 Saída esperada: filme presente na listagem	Filme com os dados informados está na listagem.	**Passou**
CT-U05	HU02/RF03,CA2	1. Instanciar CineControl. 2. Chamar cadastrar_filme(titulo=""). 3. Verificar se ErroValidacao é lançado.	Entrada: título vazio Saída esperada: ErroValidacao ("O campo Título é obrigatório")	ErroValidacao com mensagem sobre título obrigatório.	**Passou**
CT-U06	HU02/CA3	1. Instanciar CineControl. 2. Chamar cadastrar_filme(titulo="X", duracao=0). 3. Verificar se ErroValidacao é lançado.	Entrada: duração=0 Saída esperada: ErroValidacao ("A duração deve ser maior que zero")	ErroValidacao com mensagem sobre duração inválida.	**Passou**
CT-U07	HU02/RN10	1. Cadastrar filme e sessão vinculada. 2. Tentar excluir o filme. 3. Verificar se ErroValidacao é lançado.	Pré-cond: 1 sessão vinculada ao filme Saída esperada: ErroValidacao impedindo exclusão	ErroValidacao levantado — filme com sessão não pode ser excluído.	**Falhou (Bug 2)**
CT-U08	HU03/RF07	1. Instanciar CineControl. 2. Chamar cadastrar_sala(nome="Sala 1", capacidade=80). 3. Verificar se sala foi criada com capacidade=80.	Entrada: nome="Sala 1", capacidade=80 Saída esperada: sala criada com capacidade=80	Sala retornada com capacidade igual a 80.	**Passou**

CT-U09	HU03/RN04,CA2	1. Instanciar CineControl. 2. Chamar cadastrar_sala(capacidade=0). 3. Verificar se ErroValidacao é lançado.	Entrada: capacidade=0 Saída esperada: ErroValidacao ("A capacidade deve ser maior que zero")	ErroValidacao com mensagem sobre capacidade inválida.	**Passou**
CT-U10	HU04/CA4	1. Cadastrar sala com capacidade=50. 2. Cadastrar sessão vinculada à sala. 3. Verificar ingressos_disponiveis da sessão.	Entrada: sala capacidade=50 Saída esperada: sessão com ingressos_disponiveis=50	ingressos_disponiveis igual à capacidade da sala.	**Passou**
CT-U11	HU04/RN06,CA3	1. Cadastrar sala, filme e sessão. 2. Tentar cadastrar 2ª sessão com mesma sala/data/horário. 3. Verificar se ErroValidacao é lançado.	Entrada: conflito de horário na mesma sala Saída esperada: ErroValidacao de conflito	ErroValidacao levantado ao detectar conflito de horário.	**Passou**
CT-U12	HU06/CA5	1. Criar sessão com 2 ingressos disponíveis. 2. Realizar reserva de 2 ingressos. 3. Verificar ingressos_disponiveis=0.	Entrada: 2 disponíveis, reserva de 2 Saída esperada: ingressos_disponiveis=0	Disponibilidade zerada após reserva de todos os ingressos.	**Passou**
CT-U13	HU06/RN07,CA4	1. Criar sessão com 2 ingressos disponíveis. 2. Tentar reservar 5 ingressos. 3. Verificar se ErroValidacao é lançado.	Entrada: 2 disponíveis, reserva de 5 Saída esperada: ErroValidacao de indisponibilidade	ErroValidacao levantado por exceder a disponibilidade.	**Passou**
CT-U14	HU06/RN07,CA2	1. Criar sessão. 2. Tentar realizar reserva com quantidade=0. 3. Verificar se ErroValidacao é lançado.	Entrada: quantidade=0 Saída esperada: ErroValidacao ("A quantidade deve ser maior que zero")	ErroValidacao levantado para quantidade zero.	**Falhou (Bug 3)**
CT-U15	HU08/RN11,CA3	1. Criar sessão com 2 ingressos. 2. Reservar 1 ingresso (resta 1 disponível). 3. Cancelar a reserva. 4. Verificar ingressos_disponiveis=2.	Saída esperada: ingressos_disponiveis restaurados a 2	Disponibilidade retorna ao valor anterior ao cancelamento.	**Falhou (Bug 4)**

3.2 Casos de Teste de API

Framework: unittest + Flask test client. Arquivo: tests/test_api.py.

ID	Endpoint	Descrição e Passos	Entrada / Saída Esperada	Critério de Sucesso	Resultado
CT-A01	POST /api/login	1. Enviar POST com email e senha válidos. 2. Verificar status HTTP e perfil retornado.	Entrada: {email, senha} válidos Saída esperada: HTTP 200, perfil="admin"	Status 200 e perfil="admin" no corpo da resposta.	**Passou**
CT-A02	POST /api/login	1. Enviar POST com senha incorreta. 2. Verificar status HTTP e mensagem de erro.	Entrada: senha incorreta Saída esperada: HTTP 401 e campo "erro" no corpo	Status 401 e campo "erro" presente.	**Passou**

CT-A03	POST /api/filmes	1. Enviar POST com título, gênero e duração válidos. 2. Verificar status HTTP.	Entrada: título, gênero e duração válidos Saída esperada: HTTP 201	Status 201 retornado.	**Passou**
CT-A04	POST /api/filmes	1. Enviar POST com título vazio. 2. Verificar status HTTP e mensagem de erro.	Entrada: título vazio Saída esperada: HTTP 400 e mensagem de erro	Status 400 e mensagem de erro no corpo.	**Passou**
CT-A05	GET /api/filmes	1. Cadastrar 1 filme. 2. Enviar GET /api/filmes. 3. Verificar status e tamanho da lista.	Pré-cond: 1 filme cadastrado Saída esperada: HTTP 200 e lista com 1 item	Status 200 e lista contendo exatamente 1 item.	**Passou**
CT-A06	POST /api/salas	1. Enviar POST com capacidade=-5. 2. Verificar status HTTP e mensagem de erro.	Entrada: capacidade=-5 Saída esperada: HTTP 400 e mensagem de erro	Status 400 e mensagem de erro no corpo.	**Passou**
CT-A07	POST /api/sessoes	1. Enviar POST com filme_id=999 (inexistente). 2. Verificar status HTTP.	Entrada: filme_id=999 Saída esperada: HTTP 400	Status 400 retornado.	**Passou**
CT-A08	POST /api/reservas	1. Criar sessão com 1 ingresso disponível. 2. Enviar POST com quantidade=1. 3. Verificar status HTTP.	Entrada: quantidade=1 (dentro da disponibilidade) Saída esperada: HTTP 201	Status 201 retornado.	**Passou**
CT-A09	POST /api/reservas	1. Criar sessão com 1 ingresso disponível. 2. Enviar POST com quantidade=10. 3. Verificar status HTTP.	Entrada: quantidade=10 (acima da disponibilidade) Saída esperada: HTTP 400	Status 400 retornado.	**Passou**

3.3 Casos de Teste E2E

Framework: unittest + Flask test client (simulação de jornadas completas via API). Arquivo: tests/test_e2e.py. Foram adicionados dois novos fluxos E2E: CT-E02 (fluxo do administrador) e CT-E03 (fluxo de erro), conforme exigência do feedback.

ID	Fluxo	Passos Numerados	Critério de Sucesso	Resultado
CT-E01	Fluxo completo do cliente (reserva e cancelamento)	1. Login do administrador (POST /api/login). 2. Cadastro de filme (POST /api/filmes). 3. Cadastro de sala (POST /api/salas). 4. Cadastro de sessão (POST /api/sessoes). 5. Login do cliente (POST /api/login). 6. Consulta da programação (GET /api/sessoes). 7. Reserva de 2 ingressos (POST /api/reservas). 8. Consulta de "Minhas Reservas" (GET /api/reservas). 9. Verificar redução de ingressos disponíveis. 10. Cancelamento da reserva (DELETE /api/reservas/{id}). 11. Verificar devolução dos ingressos à sessão.	Todos os passos retornam status de sucesso (2xx); após o cancelamento, ingressos_disponiveis retorna ao valor antes da reserva.	**Falhou no passo 11 (Bug 4)**
CT-E02	Fluxo do administrador	1. Login do administrador (POST /api/login com perfil="admin"). 2. Cadastro de 2 filmes (POST /api/filmes). 3. Cadastro de 1 sala	Passos 1–6 retornam 2xx; passo 7 retorna HTTP 400 (bloqueio correto por RN10); passos 8–9	**Falhou no passo 7 (Bug 2)**

	(gestão completa do catálogo)	(POST /api/salas). 4. Cadastro de 2 sessões para filmes diferentes (POST /api/sessoes). 5. Listagem de todos os filmes (GET /api/filmes). 6. Listagem de todas as sessões (GET /api/sessoes). 7. Tentativa de excluir filme com sessão vinculada (DELETE /api/filmes/{id}). 8. Exclusão da sessão primeiro (DELETE /api/sessoes/{id}). 9. Exclusão do filme sem sessão vinculada (DELETE /api/filmes/{id}).	retornam 2xx com remoção efetiva.	
CT-E03	Fluxo de cenário de erro (ações inválidas do cliente)	1. Tentativa de login com credenciais inválidas (POST /api/login, senha errada). 2. Verificar rejeição com HTTP 401. 3. Login válido do cliente (POST /api/login). 4. Tentativa de reserva em sessão inexistente (POST /api/reservas, sessao_id=999). 5. Verificar rejeição com HTTP 400. 6. Cadastro de sessão com 1 ingresso disponível. 7. Tentativa de reserva com quantidade=0. 8. Verificar rejeição com HTTP 400. 9. Tentativa de reserva com quantidade=5 (acima da disponibilidade). 10. Verificar rejeição com HTTP 400.	Passo 2 retorna HTTP 401; passos 5, 8 e 10 retornam HTTP 400. Nenhuma reserva indevida é criada em nenhuma das tentativas.	**Falhou nos passos 7–8 (Bug 3)**

4. Implementação dos Testes

Os testes foram implementados em Python, utilizando exclusivamente a biblioteca padrão unittest e o cliente de testes embutido no Flask para os testes de API e E2E, o que simplifica a execução em qualquer ambiente com Python 3 instalado.

4.1 Estrutura do Repositório

- cinecontrol/core.py — regras de negócio (login, filmes, salas, sessões, reservas).
- cinecontrol/api.py — API REST (Flask) que expõe as regras de core.py.
- cinecontrol/cli.py — interface de linha de comando (frontend do protótipo).
- tests/test_unit.py — 15 testes unitários (CT-U01 a CT-U15).
- tests/test_api.py — 9 testes de API (CT-A01 a CT-A09).
- tests/test_e2e.py — 3 testes E2E (CT-E01 a CT-E03).
- README.md — instruções de instalação e execução dos testes.

4.2 Como Executar os Testes

Pré-requisito: Python 3 e Flask instalados (pip install flask).

```
python3 -m unittest discover -v tests
```

O comando acima executa automaticamente todos os testes unitários, de API e E2E, gerando o log completo no console (ver seção 5.3 para as evidências).

5. Execução e Evidências dos Testes

5.1 Resumo da Execução

A suíte completa de testes foi executada com o comando "python3 -m unittest discover -v tests". O resultado consolidado foi:

Tipo de Teste	Quantidade	Passaram	Falharam (propositais)
Unitários	15	11	4
API	9	9	0
E2E	3	0	3
TOTAL	27	20	7

As 7 falhas registradas são esperadas e propositais: evidenciam 4 defeitos inseridos deliberadamente no código. O Bug 4 (não devolução de ingressos ao cancelar) é coberto por CT-U15, CT-E01 e CT-E03, resultando em múltiplas falhas para o mesmo defeito.

5.2 Bugs Encontrados e Analisados

Nº	Local no código	Descrição do defeito	Regra violada	Casos que evidenciam
1	core.py — autenticar()	A comparação da senha ignora maiúsculas/minúsculas, permitindo login com senha em caixa diferente da cadastrada.	RN01	CT-U03
2	core.py — excluir_filme()	A verificação de sessões vinculadas foi omitida: o sistema permite excluir um filme com sessões associadas.	RN10	CT-U07, CT-E02
3	core.py — realizar_reserva()	A validação de quantidade usa condição incorreta, permitindo reservas com quantidade igual a zero.	RN07/CA2-HU06	CT-U14, CT-E03
4	core.py — cancelar_reserva()	Ao cancelar uma reserva, os ingressos não são devolvidos à disponibilidade da sessão.	RN11/CA3-HU08	CT-U15, CT-E01, CT-E03

Cada defeito foi inserido propositalmente no código-fonte (marcados com o comentário "BUG PROPOSITAL" em core.py) para demonstrar, na prática, a capacidade dos testes automatizados de detectar falhas de implementação em relação às regras de negócio.

5.3 Evidência da Execução (log completo)

A seguir, o log completo da execução da suíte de testes (saída do console), coletado em ambiente local:

```
test_ct_a01_login_valido_retorna_200 ... ok test_ct_a02_login_invalido_retorna_401 ... ok
test_ct_a03_criar_filme_valido_retorna_201 ... ok
test_ct_a04_criar_filme_sem_titulo_retorna_400 ... ok
test_ct_a05_listar_filmes_retorna_200 ... ok
test_ct_a06_criar_sala_com_capacidade_invalida_retorna_400 ... ok
test_ct_a07_criar_sessao_com_filme_inexistente_retorna_400 ... ok
test_ct_a08_reserva_valida_retorna_201 ... ok
test_ct_a09_reserva_acima_da_capacidade_retorna_400 ... ok
test_ct_e01_fluxo_completo_de_reserva_do_cliente ... FAIL
test_ct_e02_fluxo_administrador_gestao_catalogo ... FAIL test_ct_e03_fluxo_cenario_de_erro
... FAIL test_ct_u01_login_com_credenciais_validas ... ok
test_ct_u02_login_com_senha_incorreta_deve_falhar ... ok
test_ct_u03_login_deve_ser_case_sensitive_na_senha ... FAIL
```

```
test_ct_u04_cadastrar_filme_valido ... ok
test_ct_u05_cadastrar_filme_sem_titulo_deve_falhar ... ok
test_ct_u06_cadastrar_filme_com_duracao_invalida_deve_falhar ... ok
test_ct_u07_excluir_filme_com_sessao_vinculada_deve_ser_impedido ... FAIL
test_ct_u08_cadastrar_sala_valida ... ok
test_ct_u09_cadastrar_sala_com_capacidade_invalida_deve_falhar ... ok
test_ct_u10_cadastrar_sessao_define_ingressos_pela_capacidade ... ok
test_ct_u11_cadastrar_sessao_com_conflito_de_horario_deve_falhar ... ok
test_ct_u12_reserva_valida_reduz_ingressos_disponiveis ... ok
test_ct_u13_reserva_acima_da_disponibilidade_deve_falhar ... ok
test_ct_u14_reserva_com_quantidade_zero_deve_falhar ... FAIL
test_ct_u15_cancelar_reserva_deve_devolver_ingressos ... FAIL  Ran 27 tests in 0.051s
FAILED (failures=7)
```

6. Relatório Final

6.1 Introdução ao Sistema Desenvolvido

O CineControl é um sistema acadêmico de gerenciamento de cinema, implementado como um protótipo CLI em Python com API REST (Flask) e armazenamento em memória. Permite a administradores cadastrar e manter filmes, salas e sessões, e a clientes consultar a programação e realizar reservas de ingressos. O sistema foi desenhado para viabilizar a aplicação prática das técnicas de teste exigidas pela disciplina, conforme o Documento de Visão.

6.2 Planejamento de Testes (Resumo)

O planejamento definiu como objetivo validar as regras de negócio e requisitos funcionais do CineControl por meio de três níveis de teste — unitário, de API e E2E — utilizando unittest e o Flask test client. Os papéis de todos os cinco integrantes foram formalmente definidos (seção 2.5), incluindo a atuação de Igor Ferreira na implementação dos novos testes E2E.

6.3 Descrição dos Casos de Teste (Resumo)

Foram elaborados 27 casos de teste no total: 15 unitários (cobrindo login, gerenciamento de filmes, salas, sessões e reservas), 9 de API (cobrindo os principais endpoints REST do backend) e 3 E2E. Os três fluxos E2E cobrem: (1) o fluxo completo do cliente (login, reserva e cancelamento); (2) o fluxo do administrador (gestão do catálogo, incluindo cenário de bloqueio de exclusão); e (3) o fluxo de cenário de erro (rejeição de ações inválidas). Todos os casos possuem passos numerados e critério de sucesso explícito.

6.4 Resultados das Execuções e Evidências

Da suíte de 27 testes automatizados, 20 passaram e 7 falharam. Todas as falhas são propositais, associadas a 4 defeitos inseridos deliberadamente no código para demonstração das técnicas de teste. O log completo de execução está na seção 5.3 e os defeitos estão detalhados na seção 5.2.

6.5 Conclusão e Lições Aprendidas

A aplicação prática de testes unitários, de API e E2E sobre o CineControl demonstrou, de forma concreta, o valor dessas técnicas na verificação e validação de software. Os 27 casos de teste elaborados a partir do Documento de Visão e das Histórias de Usuário foram capazes de detectar 100% dos defeitos inseridos propositalmente.

Como principais lições aprendidas, o grupo destaca:

- A importância de derivar os casos de teste diretamente dos critérios de aceitação das Histórias de Usuário, tornando a cobertura mais aderente aos requisitos reais do sistema.
- O valor complementar dos três níveis de teste: os bugs de cancelamento (Bug 4) e de bloqueio de exclusão (Bug 2) foram detectados tanto nos testes unitários isolados quanto nos fluxos E2E, reforçando a confiança na cobertura.
- A importância de descrever o sistema como ele realmente é: o alinhamento entre o texto do relatório e a arquitetura real do protótipo (CLI + memória, sem banco de dados) evita confusão na avaliação e na manutenção do projeto.
- A viabilidade de manter uma suíte de testes simples e sem dependências externas, facilitando a execução em qualquer ambiente.
- A necessidade de distribuir as contribuições entre todos os integrantes, com papel formal definido para cada um — garantindo equidade na autoria e rastreabilidade no controle de versão.

7. Link para o Repositório no GitHub

https://github.com/victorlimapy/CineControl-Test

O repositório contém o código-fonte completo (core.py, api.py, cli.py), os três arquivos de testes (test_unit.py, test_api.py, test_e2e.py), o README.md com instruções de execução e este Relatório Final.